

Theoretical Computer Science Endterm test, December 2, 2023

The exam consists of **4 exercises**. Available time: 1 hr 30 min.; you may write your answers in Italian, if you wish.

All exercises must be addressed for the written exam to be considered sufficient.

F.NAME L.NAME PERSON CODE

Exercise 1 (8 pts)

1. Consider the language

$$L = \{ a^m b^{2k} c^n \mid m > 0 \wedge n > 0 \wedge (n = m \vee n = m+1) \wedge k \geq 0 \}$$

For instance, strings $abbcc$, $ac \in L$, $abc \notin L$.

- a) Design a minimal-power automaton that accepts language L (solutions with fewer states are preferred).
- b) Write computations of the automaton that accept the string $abbc$ and acc , and one that rejects the string abc .

2. Consider the language

$$L = \{ x \in \{ a, b \}^* \mid |x| \bmod 2 = 0 \wedge \#_a(x) \bmod 2 = 1 \}$$

where the operator MOD denotes the remainder of the integer division, and $\#_a(x)$ denotes the number of characters a included in string x . For instance, strings ab , ba , $babaab \in L$, while strings ε , b , aba , $abab \notin L$.

- a) Define the indistinguishability relation ' $\approx_L$ ' for the strings of the language.
- b) List all the equivalence classes defined by the above indistinguishability relation.
- c) Based on the above identified indistinguishability relation ' $\approx_L$ ' and its equivalence classes, design a finite state automaton that accepts language L .

Exercise 2 (8 pts)

Consider a subprogram *greaterDouble* (a, n, k, u) whose parameters have the following meaning

a is an input array of integer numbers that includes at least 3 elements

n is the input actual number of elements of array a (notice that indices inside array a start from 1)

k is an input index position inside array a , strictly included between the first and last position

u is a Boolean output, that is a Boolean result returned to the caller

In implementing the subprogram one can assume that all the elements of array a , from the initial position up to index k not included, are strictly less than all those stored in the succeeding positions, starting from position k included up to the last position. For instance, the parameter values

$a = [2, 3, 5, 9, 7]$ $n = 5$ $k = 3$ satisfy this requirement, while

$a = [7, 4, 5, 9, 6]$ $n = 5$ $k = 3$ do not satisfy it.

After execution of the subprogram the output parameter u must be true if and only if

for every value stored in array a from the initial position up to index k not included an integer equal to *twice* that value is stored in some of the succeeding positions, starting from index k included up to the last position.

Examples:

when $a = [2, 3, 13, 6, 4]$, $n = 5$, $k = 3$, $u = \text{true}$ then the program behavior is correct (u is correctly true because 2, 3 are stored in the positions < 3 , and 4, 6 are stored in some of the positions ≥ 3).

when $a = [2, 3, 13, 5, 4]$, $n = 5$, $k = 3$, $u = \text{false}$ then the program behavior is correct (u is correctly false because 2, 3 are stored in the positions < 3 , and the value $6=2 \times 3$, is not stored in any of the positions ≥ 3).

- Write mathematical logic formulas for the pre- and post-conditions of program *greaterDouble*
- For each of the following sets of values of the parameters of the *greaterDouble* subprogram, say if the precondition and postcondition are satisfied, and if the parameter values are compatible with a correct implementation of the subprogram with respect to its specification by means of the pre- and post-condition. Shortly justify your answer.

1) $a = [4, 3, 6, 7, 8]$, $n = 5$, $k = 3$, $u = \text{false}$

Precondition satisfied (Y/N): ...

Postcondition satisfied (Y/N): ...

Parameter values compatible with correct implementation (Y/N): ...

Short justification: ...

2) $a = [8, 3, 5, 9, 7]$, $n = 5$, $k = 3$, $u = \text{true}$

Precondition satisfied (Y/N): ...

Postcondition satisfied (Y/N): ...

Parameter values compatible with correct implementation (Y/N): ...

Short justification: ...

3) $a = [4, 3, 5, 6, 8]$, $n = 5$, $k = 3$, $u = \text{true}$

Precondition satisfied (Y/N): ...

Postcondition satisfied (Y/N): ...

Parameter values compatible with correct implementation (Y/N): ...

Short justification: ...

4) $a = [2, 3, 5, 9, 7]$, $n = 5$, $k = 3$, $u = \text{false}$

Precondition satisfied (Y/N): ...

Postcondition satisfied (Y/N): ...

Parameter values compatible with correct implementation (Y/N): ...

Short justification: ...

5) $a = [4, 3, 5, 7, 8]$, $n = 5$, $k = 3$, $u = \text{true}$

Precondition satisfied (Y/N): ...

Postcondition satisfied (Y/N): ...

Parameter values compatible with correct implementation (Y/N): ...

Short justification: ...

6) $a = [8, 3, 5, 9, 7]$, $n = 5$, $k = 3$, $u = \text{false}$

Precondition satisfied (Y/N): ...

Postcondition satisfied (Y/N): ...

Parameter values compatible with correct implementation (Y/N): ...

Short justification: ...

Exercise 3 (8 pts)

Answer the following questions, providing clear, concise justifications.

1. Is the set $S_1 = \{ x \mid x \text{ is the sum of two prime numbers} \}$ decidable? Is it semidecidable?
2. Is the set $S_2 = \{ x \mid \forall y (f_x(y) = 1 \text{ if } y \text{ is the sum of two prime numbers; } f_x(y) = 0 \text{ otherwise}) \}$ decidable?
3. Say if the following problem is solvable. Given a generic regular grammar G_R , is the language generated by G_R infinite?
4. Say if the following problem is solvable. Given a generic general unrestricted grammar G_U , is the language generated by G_U infinite?

Exercise 4 (8 pts)

The following RAM program reads from the input a natural number x and computes and writes on the output the value $2^{(x^2)}$. Notice that the use of the memory cells is as follows

M[1] input x M[2] counter M[3] cumulated product

	READ 0	read x
	STORE 1	M[1] = x
	STORE 2	
	MUL 2	
	STORE 2	counter = x^2
	LOAD =1	
	STORE 3	cumulated product = 1
LP:	LOAD 2	
	JZ WRT	go to write if counter = 0
	SUB = 1	
	STORE 2	counter decreased by 1
	LOAD 3	cumulated product multiplied by 2
	MUL =2	
	STORE 3	
	JUMP LP	
WRT:	WRITE 3	
	HALT	

1. Determine the time and space complexity class for this program, as a function of the input value x , according to the uniform cost criterion. Briefly justify your answer.
2. Determine the time and space complexity class for this program, as a function of the input value x , according to the logarithmic cost criterion. Briefly justify your answer.
3. Consider the string s that in the input encodes the input value x (you may assume that x is encoded in binary). Determine, according to the logarithmic cost criterion, the time and space complexity class for this program, as a function of the length $n = |s|$ of the input string s . Briefly justify your answer.

Solutions

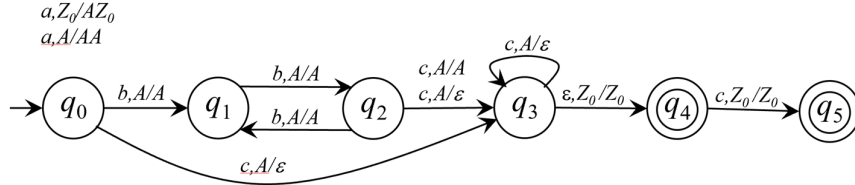
Exercise 1

1. Consider the language

$$L = \{ a^m b^{2k} c^n \mid m > 0 \wedge n > 0 \wedge (n = m \vee n = m+1) \wedge k \geq 0 \}$$

For instance, strings $abbcc$, $ac \in L$, $abc \notin L$.

- a) Design a minimal-power automaton that accepts language L (solutions with fewer states are preferred). A nondeterministic PDA suffices.



- b) Write computations of the automaton that accept the string $abbcc$ and acc , and one that rejects the string abc . We refer to the first of the two above automata.
- $\langle abbcc, q_0, Z_0 \rangle \vdash \langle bbc, q_0, AZ_0 \rangle \vdash \langle bc, q_1, AZ_0 \rangle \vdash \langle c, q_2, AZ_0 \rangle \vdash \langle \varepsilon, q_3, Z_0 \rangle \vdash \langle \varepsilon, q_4, Z_0 \rangle$ accept
- $\langle acc, q_0, Z_0 \rangle \vdash \langle cc, q_0, AZ_0 \rangle \vdash \langle c, q_3, Z_0 \rangle \vdash \langle c, q_4, Z_0 \rangle \vdash \langle \varepsilon, q_5, Z_0 \rangle$ accept
- $\langle abc, q_0, Z_0 \rangle \vdash \langle bc, q_0, AZ_0 \rangle \vdash \langle c, q_1, AZ_0 \rangle$ reject

2. Consider the language

$$L = \{ x \in \{ a, b \}^* \mid |x| \bmod 2 = 0 \wedge \#_a(x) \bmod 2 = 1 \}$$

- a) Define the indistinguishability relation ' $\approx_L$ ' for the strings of the language.

$$x \approx_L y \quad \text{iff} \quad (|x| \bmod 2 = |y| \bmod 2) \wedge (\#_a(x) \bmod 2 = \#_a(y) \bmod 2)$$

- b) List all the equivalence classes defined by the above indistinguishability relation.

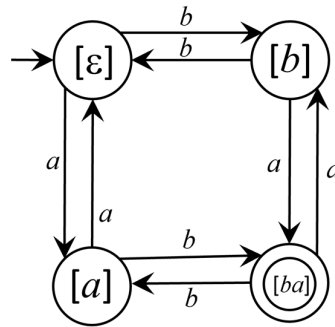
$$[\varepsilon] = \{ x \mid |x| \bmod 2 = 0 \wedge \#_a(x) \bmod 2 = 0 \}$$

$$[a] = \{ x \mid |x| \bmod 2 = 1 \wedge \#_a(x) \bmod 2 = 1 \}$$

$$[b] = \{ x \mid |x| \bmod 2 = 1 \wedge \#_a(x) \bmod 2 = 0 \}$$

$$[ba] = \{ x \mid |x| \bmod 2 = 0 \wedge \#_a(x) \bmod 2 = 1 \}$$

- c) Based on the above identified indistinguishability relation ' $\approx_L$ ' and its equivalence classes, design a finite state automaton that accepts language L .



Exercise 2

Precondition:

$$n \geq 3 \wedge 1 < k < n \wedge \forall i (1 \leq i < k \rightarrow \forall j (k \leq j \leq n \rightarrow a[i] < a[j]))$$

Postcondition:

$$u \leftrightarrow \forall i (1 \leq i < k \rightarrow \exists j (k \leq j \leq n \wedge a[j] = 2 \times a[i]))$$

- 1) $a = [4, 3, 6, 7, 8]$, $n = 5$, $k = 3$, $u = \text{false}$

Precondition satisfied (Y/N): **Y**

Postcondition satisfied (Y/N): **N**

Parameter values compatible with correct implementation (Y/N): **N**

precondition satisfied, postcondition not satisfied (u should be true), implementation incorrect

2) $a = [8, 3, 5, 9, 7]$, $n = 5$, $k = 3$, $u = \text{true}$

Precondition satisfied (Y/N): **N**

Postcondition satisfied (Y/N): **N**

Parameter values compatible with correct implementation (Y/N): **Y**

NB: precondition is **not** satisfied (value **8** greater than 5 and 7), implementation correct independently of the value of u

3) $a = [4, 3, 5, 6, 8]$, $n = 5$, $k = 3$, $u = \text{true}$

Precondition satisfied (Y/N): **Y**

Postcondition satisfied (Y/N): **Y**

Parameter values compatible with correct implementation (Y/N): **Y**

precondition and postcondition both satisfied, implementation correct

4) $a = [2, 3, 5, 9, 7]$, $n = 5$, $k = 3$, $u = \text{false}$

Precondition satisfied (Y/N): **Y**

Postcondition satisfied (Y/N): **Y**

Parameter values compatible with correct implementation (Y/N): **Y**

precondition and postcondition both satisfied (u is correctly set to false), implementation correct

5) $a = [4, 3, 5, 7, 8]$, $n = 5$, $k = 3$, $u = \text{true}$

Precondition satisfied (Y/N): **Y**

Postcondition satisfied (Y/N): **N**

Parameter values compatible with correct implementation (Y/N): **N**

precondition satisfied, postcondition not satisfied (no value at position ≥ 3 is equal to 3×2 , hence u should be false), implementation incorrect

6) $a = [8, 3, 5, 9, 7]$, $n = 5$, $k = 3$, $u = \text{false}$

Precondition satisfied (Y/N): **N**

Postcondition satisfied (Y/N): **Y**

Parameter values compatible with correct implementation (Y/N): **Y**

NB: precondition is **not** satisfied (value **8** greater than 5 and 7), implementation correct independently of the value of u

Exercise 3

Answer the following questions, providing clear, concise justifications.

1. Is the set $S1 = \{x \mid x \text{ is the sum of two prime numbers}\}$ decidable? Is it semidecidable?

Yes, $S1$ is decidable, and hence semidecidable, because it is possible (and in fact easy) to write a procedure that, given any natural number, checks if it is the sum of two prime numbers.

2. Is the set $S2 = \{x \mid \forall y (f_x(y) = 1 \text{ if } y \text{ is the sum of two prime numbers; } f_x(y) = 0 \text{ otherwise})\}$ decidable?

$S2$ is not decidable because it satisfies the hypothesis of the Rice theorem. By taking

$F = \{f \mid \forall y (f(y) = 1 \text{ if } y \text{ is the sum of two prime numbers; } f(y) = 0 \text{ otherwise})\}$,

we have $S2 = \{ x \mid f_x \in F \}$

3. Say if the following problem is solvable. Given a generic regular grammar G_R , is the language generated by G_R infinite?

The problem is solvable, because it can be reduced to the problem of determining if the language accepted by a finite state automaton is infinite, which is notoriously solvable. In fact, any regular grammar can be algorithmically translated into an equivalent finite state automaton.

4. Say if the following problem is solvable. Given a generic general unrestricted grammar G_U , is the language generated by G_U infinite?

The problem is unsolvable. In fact, given any Turing machine M , an equivalent grammar G_M can be algorithmically obtained, from M , that generates $L(M)$, the language accepted by M . Consider now the problem of determining, for a generic Turing machine M , if $L(M)$ is infinite: this problem is notoriously unsolvable. If the problem of determining if the language generated by a grammar is infinite was solvable, then the problem of determining, for a generic Turing machine M , if $L(M)$ is infinite would also be solvable, because it can be reduced to it.

Exercise 4

1. Determine the time and space complexity class for this program, as a function of the input value x , according to the uniform cost criterion.

The loop is executed x^2 times and it consists of a fixed number of instructions, therefore the time complexity is $\Theta(x^2)$. The space complexity is $\Theta(1)$ since a constant number of memory cells is used.

2. Determine the time and space complexity class for this program, as a function of the input value x , according to the logarithmic cost criterion.

The three dominating instructions for what concerns the time complexity with the logarithmic cost criterion are the following ones, for which we indicate the complexity

LOAD 3	$l(3) + l(2^{i-1})$
MUL =2	$l(2) + l(2^{i-1})$
STORE 3	$l(3) + l(2^i)$

Therefore the time complexity is

$$\sum_{i=1}^{x^2} 2 \cdot l(2^i) = \sum_{i=1}^{x^2} 2i \cdot l(2) = 2 \cdot l(2) \sum_{i=1}^{x^2} i \in \Theta(x^4)$$

The space complexity is determined by the largest value stored, $2^{(x^2)}$, therefore it is $\Theta(l(2^{(x^2)})) = \Theta(x^2)$.

3. Consider the string s that in the input encodes the input value x (you may assume that x is encoded in binary). Determine, according to the logarithmic cost criterion, the time and space complexity class for this program, as a function of the length $n = |s|$ of the input string s .

The following relations hold: $n = \log x$ and $x = 2^n$, therefore the complexity figures with respect to the parameter n are as follows. For the time complexity: $\Theta((2^n)^4) = \Theta(2^{4n})$; for the space complexity: $\Theta((2^n)^2) = \Theta(2^{2n})$.